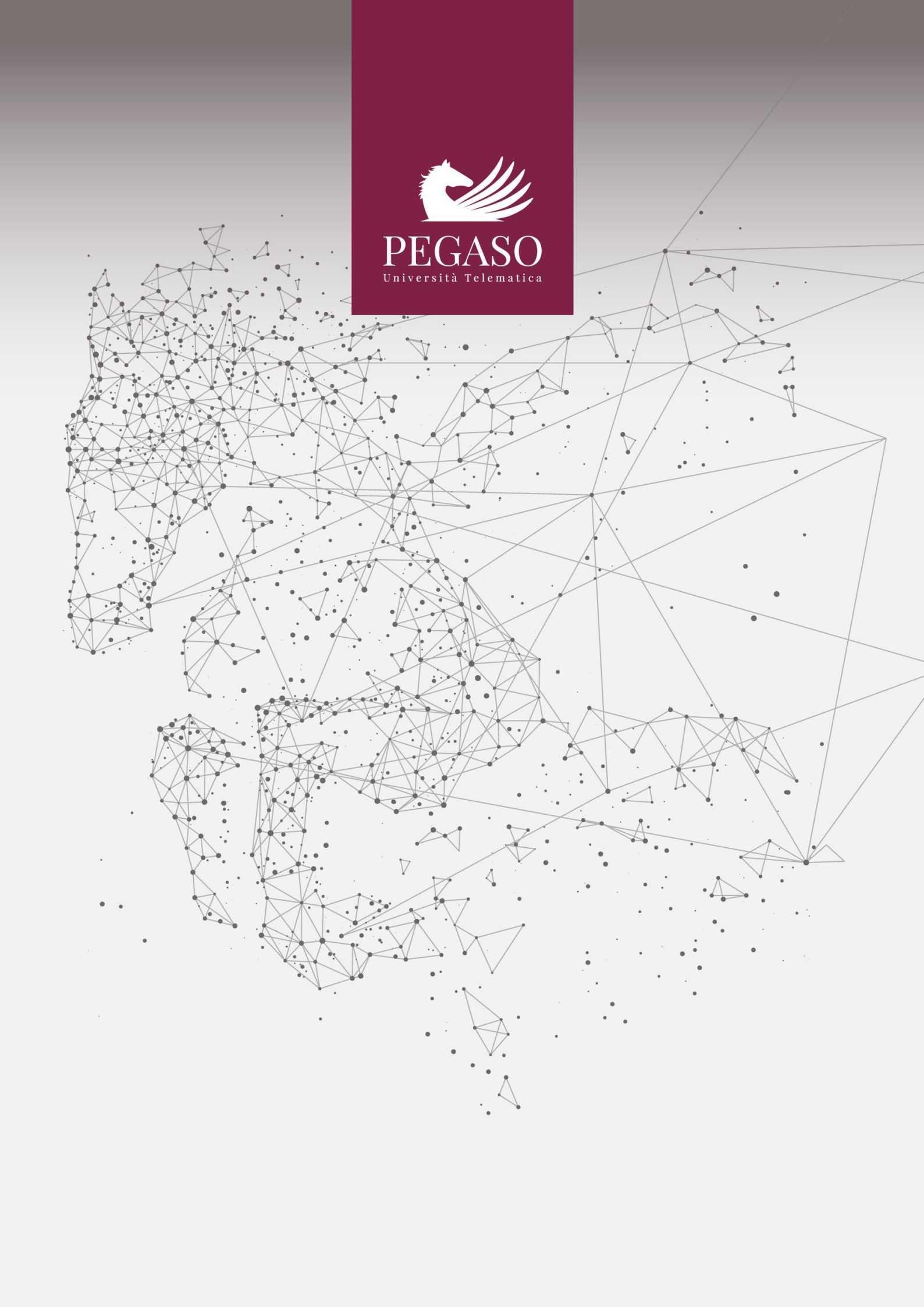

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. THREAD NEI SISTEMI NON DISTRIBUITI 5
3. IMPLEMENTAZIONE DEI THREAD 8
BIBLIOGRAFIA 10

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

Anche se i processi sono una componente essenziale dei sistemi distribuiti, in pratica è stato dimostrato che la granularità dei processi fornita dai sistemi operativi su cui sono costruiti i sistemi distribuiti è insufficiente. Invece, è stato scoperto che avere una granularità più fine aggiungendo di più thread di controllo per ogni processo rende molto più semplice costruire applicazioni distribuite e comporta prestazioni migliorate. In questa lezione daremo uno sguardo più approfondito al ruolo che i thread svolgono nei sistemi distribuiti e spiegheremo perché sono di così vitale importanza.

È importante capire cos'è un processo e la relazione tra processi e thread prima di tentare di comprendere la funzione che i thread svolgono nei sistemi informatici distribuiti. Per eseguire le istruzioni di un programma, un sistema operativo produrrà un numero di processori virtuali, ognuno dei quali eseguirà le istruzioni di un programma diverso. Il sistema operativo dispone di una tabella dei processi che contiene voci per memorizzare i valori del registro CPU, le mappe di memoria, le informazioni sui file aperti, le informazioni di accounting, i privilegi e altri dati rilevanti al fine di mantenere un record di questi processori virtuali. Queste voci, se combinate, costituiscono il contesto di processo.

È possibile pensare a un contesto di processo come all'equivalente software del contesto del processore nell'hardware. Quest'ultimo è costituito dal minimo indispensabile di informazioni che vengono automaticamente salvate dall'hardware per gestire un interrupt e per riprendere successivamente l'elaborazione dal punto in cui è stato interrotto. Come minimo, il contatore di programma (Program Counter) fa parte del contesto del processore. Tuttavia, in alcuni casi, include anche valori di altri registri, ad esempio il puntatore dello stack.

Un processo viene spesso definito come un programma in esecuzione o, più specificamente, un programma attualmente in esecuzione su uno dei processori virtuali forniti dal sistema operativo. Il fatto che il sistema operativo faccia molta attenzione a garantire che i processi indipendenti non possano influenzare intenzionalmente o accidentalmente la correttezza del comportamento reciproco è un aspetto importante che dovrebbe essere preso in considerazione. In altre parole, viene chiarito che più processi potrebbero utilizzare la stessa unità di elaborazione centrale (CPU) e altre risorse hardware contemporaneamente. Nella maggior parte dei casi, il sistema operativo necessita del supporto dell'hardware sottostante per applicare questa separazione.

Il mantenimento di questo livello di trasparenza della concorrenza comporta un costo. Ad esempio, ogni volta che viene avviato un nuovo processo, il sistema operativo è tenuto a generare un nuovo spazio di indirizzi completamente separato da qualsiasi indirizzo utilizzato in precedenza. Nel contesto della

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

memoria, "allocazione" può significare "inizializzazione di segmenti di memoria" in vari modi, tra cui l'impostazione di uno stack per i dati temporanei, l'azzeramento di un segmento di dati e la copia del programma associato in un segmento di testo. Allo stesso modo, anche la commutazione dell'unità di elaborazione centrale (CPU) tra due processi diversi può richiedere un certo sforzo. Oltre a salvare i dati nello stato corrente in quanto sono memorizzati in una varietà di registri (inclusi il program counter e il puntatore dello stack), il sistema operativo dovrà modificare i registri dell'unità di gestione della memoria (MMU) e invalidare le cache di conversione degli indirizzi come quelle trovate nel translation lookaside buffer (TLB). Inoltre, se il sistema operativo è in grado di eseguire contemporaneamente più processi di quanti ne abbia spazio nella memoria principale, potrebbe essere necessario spostare i processi avanti e indietro tra la memoria principale e il disco prima che il passaggio possa effettivamente aver luogo.

Un thread è analogo a un processo in quanto è responsabile dell'esecuzione indipendente della propria sezione di codice. A differenza dei processi, tuttavia, non viene fatto alcun tentativo di ottenere un elevato grado di trasparenza della concorrenza se ciò comporterebbe una riduzione delle prestazioni. Per questo motivo, un sistema di thread in genere memorizza solo il minimo indispensabile di informazioni per consentire a più thread di condividere una singola CPU. In particolare, il contesto di thread è in genere costituito da nient'altro che il contesto del processore, insieme ad altre informazioni per la gestione dei thread. Ad esempio, un sistema di thread può tenere traccia del fatto che un thread è attualmente bloccato su una variabile mutex per evitare di selezionare quel thread per l'esecuzione. Ciò impedisce che il thread venga scelto a caso. Nella maggior parte dei casi, le informazioni che non sono strettamente necessarie per la gestione di più thread vengono ignorate. Per questo motivo, la responsabilità di proteggere i dati dall'accesso non autorizzato da parte di diversi thread che operano all'interno di un singolo processo ricade esclusivamente sulle spalle degli sviluppatori dell'applicazione. Di conseguenza, possiamo vedere che un contesto del processore è contenuto all'interno di un contesto di thread e che un contesto di thread, a sua volta, è contenuto all'interno di un contesto di processo.

Quando si tratta di distribuire i thread nel modo appena delineato, ci sono due implicazioni significative. Per iniziare, le prestazioni di un'applicazione multithread non devono quasi mai essere peggiori di quelle del suo analogo a thread singolo. Questo è quasi sempre il caso. La maggior parte delle volte, infatti, il multithreading si traduce in un aumento delle prestazioni per l'applicazione. In secondo luogo, la creazione di applicazioni multithread richiede un ulteriore lavoro mentale perché i thread non sono automaticamente protetti l'uno contro l'altro allo stesso modo dei processi. Progettare un buon design e cercare di mantenere le cose il più semplici possibili sono sempre molto utili. Purtroppo, il modo in cui questo principio viene attuato nella pratica non dimostra che sia altrettanto ben compreso a livello generale.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Thread nei sistemi non distribuiti

L'adozione diffusa di thread può essere attribuita alla proliferazione dei numerosi vantaggi offerti dai processi multithread. Il vantaggio più significativo deriva dal fatto che, in un processo con un singolo thread, l'intero processo viene interrotto ogni volta che viene eseguita una chiamata di sistema bloccante. Per dare un'idea del significato di questo, supponiamo di avere un'applicazione come un programma di fogli di calcolo e assumeremo che l'utente voglia cambiare i valori in modo continuo e interattivo. Una delle caratteristiche più importanti di un programma per fogli di calcolo è che può preservare le dipendenze funzionali che esistono tra le varie celle, spesso provenienti da più fogli di calcolo. Di conseguenza, ogni volta che una cella viene modificata, anche tutte le celle che dipendono da essa vengono aggiornate automaticamente. Un singolo utente che modifica il valore in una singola cella può causare l'attivazione di un numero elevato di calcoli a seguito della modifica. Quando è presente un solo thread di controllo, il calcolo non può procedere mentre il programma è in attesa di input. Allo stesso modo, fornire input mentre vengono calcolate le dipendenze può essere un compito difficile. La risposta semplice è implementare almeno due diversi thread di controllo, uno dei quali è responsabile della gestione dell'interazione con l'utente e l'altro è responsabile dell'aggiornamento del foglio di calcolo. Mentre gli altri due thread funzionano, un terzo thread potrebbe essere utilizzato per eseguire il backup del foglio di calcolo su disco mentre viene lavorato dagli altri due thread.

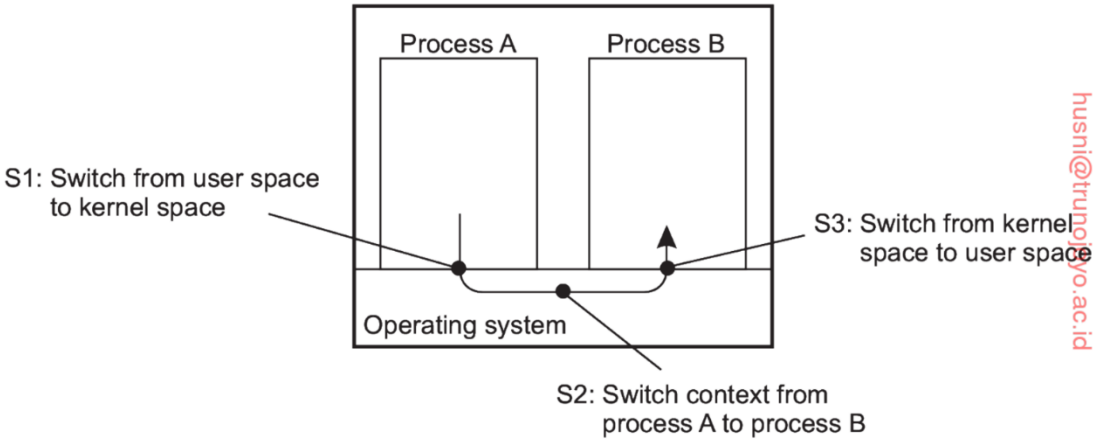
Quando un programma viene eseguito su un computer con più processori o core, il multithreading consente di sfruttare il calcolo parallelo, che è un altro dei suoi numerosi vantaggi. In questo scenario, ogni thread viene mappato su una CPU o su un core distinto e i dati condivisi vengono mantenuti in una memoria principale accessibile da tutti i thread. Tale parallelismo può essere trasparente quando è progettato correttamente, il che significa che il processo funzionerà altrettanto bene su un sistema con un singolo processore, anche se più lentamente. Con la disponibilità di computer multiprocessore e multicore che non sono eccessivamente costosi, il multithreading per il parallelismo sta diventando una tecnica sempre più importante. Tali sistemi informatici sono tipicamente utilizzati per l'esecuzione di server in applicazioni client-server; tuttavia, ora sono ampiamente utilizzati anche in dispositivi come gli smartphone. Questo perché tali sistemi informatici sono estremamente affidabili e scalabili.

L'uso del multithreading può anche essere utile quando si tratta del contesto di applicazioni di grandi dimensioni. Questi tipi di applicazioni sono in genere sviluppati come una raccolta di programmi cooperanti, ognuno dei quali è destinato ad essere eseguito dal proprio processo. Questo metodo viene spesso utilizzato in ambienti basati su Unix. I meccanismi di comunicazione interprocesso (IPC) sono ciò che

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

consente di mettere in atto la cooperazione tra diversi programmi. Pipeline, code di messaggi e segmenti di memoria condivisa sono esempi tipici di questi meccanismi nei sistemi Unix. Lo svantaggio più significativo condiviso da tutti i meccanismi IPC è la necessità di un cambio di contesto (context switch) relativamente impegnativo durante la comunicazione. Ciò è illustrato nella Figura 1.



husni@trunojoyo.ac.id

Figura 1 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

A causa del fatto che IPC richiede l'intervento del kernel, un processo in genere dovrà prima passare dalla modalità utente alla modalità kernel. Questa transizione è rappresentata da S1 nella Figura 1. A tale scopo, è necessario aggiornare la mappa di memoria nella MMU e svuotare il TLB. Un context switch di processo avviene all'interno del kernel (S2 nella figura), dopo di che l'altra parte può essere attivata passando nuovamente dalla modalità kernel alla modalità utente (S3 in Figura 1). Quest'ultimo switch richiede ancora una volta la modifica della mappa MMU e il lavaggio del TLB.

Un'applicazione può anche essere costruita in modo tale che thread distinti siano responsabili dell'esecuzione dei suoi vari componenti come alternativa all'utilizzo dei processi. L'uso di dati condivisi gestisce tutti gli aspetti della comunicazione tra questi componenti. In alcune implementazioni, il passaggio da un thread all'altro può avvenire interamente nello spazio utente. In altre implementazioni, tuttavia, il kernel è a conoscenza dei thread e li pianifica. Il risultato può essere un aumento significativo del proprio livello di prestazioni.

Ultimo ma non meno importante, c'è una pura ragione di ingegneria del software per utilizzare i thread, ovvero che molte applicazioni sono molto più semplici da strutturare se viste come una raccolta di

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

thread cooperanti. Si pensi alle applicazioni che devono eseguire una serie di attività diverse che sono più o meno indipendenti l'una dall'altra, come il foglio di calcolo di esempio che abbiamo visto in precedenza.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Implementazione dei thread

La creazione di thread avviene in genere sotto forma di pacchetto di thread. Un pacchetto come questo include operazioni per creare e distruggere thread, oltre a operazioni su variabili di sincronizzazione come mutex e variabili di condizione. Quando si tratta dell'implementazione effettiva di un pacchetto di thread, ci sono fondamentalmente due approcci diversi. Il primo metodo prevede la creazione di una libreria di thread che opera esclusivamente all'interno dell'ambiente di lavoro dell'utente. La seconda strategia prevede che il kernel venga reso consapevole dei thread in modo che possa pianificarli.

Una libreria di thread a livello utente offre una serie di vantaggi. In primo luogo, non è costoso avviare e terminare i thread. Il costo di creazione di un thread è determinato principalmente dalla quantità di memoria che deve essere allocata per costruire uno stack di thread. Ciò è dovuto al fatto che tutta l'amministrazione dei thread è memorizzata nello spazio degli indirizzi dell'utente. In modo simile, il processo di distruzione di un thread comporta principalmente la liberazione di memoria per lo stack, che non è più in uso. Entrambe queste procedure hanno costi bassi.

La possibilità di cambiare il contesto di un thread può spesso essere realizzata con poche istruzioni grazie a un secondo vantaggio dei thread a livello utente. In poche parole, solo i valori dei registri della CPU devono essere salvati e quindi ricaricati con i valori precedentemente salvati del thread. Non vi è alcun motivo valido per modificare le mappe di memoria, cancellare il TLB o eseguire operazioni di accounting della CPU. Quando due thread diversi devono sincronizzarsi tra loro, ad esempio quando si inserisce una sezione di dati condivisi, il contesto di uno dei thread verrà cambiato. D'altra parte, una parte significativa del sovraccarico causato dal cambio di contesto è dovuto alla perturbazione delle cache di memoria.

Quando si utilizzano thread a livello utente, viene applicato il modello di threading molti-a-uno, il che significa che più thread vengono mappati a una singola entità schedulabile. Quando viene effettuata una chiamata di sistema bloccante, il risultato è che l'intero processo a cui appartiene il thread, così come tutti gli altri thread in quel processo, verrà immediatamente arrestato. Come è stato spiegato in precedenza, i thread sono uno strumento estremamente utile per partizionare applicazioni di grandi dimensioni in sottocomponenti che possono essere eseguiti logicamente in modo simultaneo. In tal caso, altre parti dovrebbero comunque poter essere eseguite nel frattempo anche se le operazioni di I/O vengono bloccate. I thread creati a livello utente sono inutili per applicazioni come queste.

L'implementazione dei thread nel kernel del sistema operativo produce quello che è noto come il modello di threading uno-a-uno, in cui ogni singolo thread è un'entità schedulabile. Ogni operazione di thread, come la creazione, la cancellazione, la sincronizzazione e così via, dovrà essere eseguita dal kernel,

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

il che significa che sarà necessaria una chiamata di sistema. È possibile che il passaggio da un contesto di thread all'altro diventi presto altrettanto costoso del passaggio da un contesto di processo all'altro. Tuttavia, alla luce del fatto che le prestazioni del cambio di contesto sono generalmente determinate da un uso inefficiente delle cache di memoria e non dalla distinzione tra il modello di threading molti-a-uno e uno-a-uno, molti sistemi operativi offrono ora quest'ultimo modello, se non altro per semplicità.

Come ultimo punto da considerare, è essenziale essere consapevoli del fatto che l'organizzazione delle esecuzioni simultanee e concorrenti che si verificano all'interno di un'applicazione è possibile tramite l'uso di thread. In realtà, osserviamo frequentemente che le applicazioni sono costruite come una raccolta di processi concorrenti, che lavorano tutti insieme per sfruttare le funzionalità interprocesso fornite da un sistema operativo. L'organizzazione del server Web Apache, che, per impostazione predefinita, inizia con una manciata di processi per la gestione delle richieste in arrivo, è un buon esempio di questa strategia e serve come esempio della sua applicazione. Tuttavia, nonostante abbia un solo thread, ogni processo è ancora in grado di comunicare con altre istanze del server utilizzando metodi abbastanza standard.

L'utilizzo di processi piuttosto che thread ha il vantaggio significativo di separare lo spazio dei dati. Ciò significa che ogni processo funziona sulla propria sezione di dati pur essendo protetto da interferenze da altri processi da parte del sistema operativo. Il vantaggio di questa separazione non dovrebbe essere sottovalutato. La programmazione dei thread è notoriamente difficile perché il programmatore è l'unico responsabile della gestione dell'accesso simultaneo ai dati condivisi. Ciò rende la separazione di questi due processi un vantaggio significativo. Attraverso l'uso di processi, gli spazi dati sono in ultima analisi salvaguardati dal supporto dell'hardware. L'hardware genererà un'eccezione se un processo tenta di accedere ai dati che si trovano all'esterno della memoria a cui è stato allocato e l'eccezione verrà quindi elaborata dal sistema operativo. Non è disponibile tale supporto per più thread che operano contemporaneamente all'interno dello stesso processo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.
- Robbins K. and Robbins S. UNIX Systems Programming. Prentice Hall, Englewood Cliffs, N.J., 2003
- Stevens W. UNIX Network Programming – Interprocess Communication. Prentice Hall, Englewood Cliffs, N.J., 2nd edition, 1999.
- Srinivasan S. Kilim: A Server Framework with Lightweight Actors, Isolation Types and Zero-Copy Messaging. Ph.d., University of Cambridge, Computer Laboratory, Feb. 2010.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).